Valve Inventory

Installation Manual

This covers getting the tool running from nothing - whether you're setting it up for the first time, or you were handed an exported copy by someone else (see File > Export archive and tools in the app, or QUICKSTART.md). **Already have an installation and moving to a newer version instead? See the Upgrade Guide, not this document** - the procedure for keeping your existing collection is different from a from-scratch setup.

1. Requirements

- **Python 3.8 or later.** Check with `python3 --version`.
- **tkinter**, for the desktop window. Ships with most Python installs; on Debian/Ubuntu it's a separate package: `sudo apt install python3-tk`.
- **openpyxl**, only if you want to export a spreadsheet snapshot: `pip install openpyxl`.
- **reportlab**, only if you want to regenerate these manuals: `pip install reportlab`.

> **Note:** Nothing else is required at runtime. The database engine (SQLite) is built into Python's standard library - there is no server to install or configure.

2. Getting the files

The repository is `GarethDaviesLondon/ValveInventory` on GitHub: `https://github.com/GarethDaviesLondon/ValveInventory`. Three ways to get a working copy:

2a. Clone the repository (latest)

Tracks ongoing development on the main branch.

```
git clone https://github.com/GarethDaviesLondon/ValveInventory.git
cd ValveInventory
```

2b. Download a tagged release (recommended once one exists)

A tagged release is a known-good snapshot rather than whatever main happens to be at the moment - the first is planned as **v1.0**. Once it's out:

- **No git needed** - open `https://github.com/GarethDaviesLondon/ValveInventory/releases`, pick the release (e.g. v1.0), and download its “Source code (zip)” asset under Assets. Unzip it and follow section 3 below as normal.
- **With git** - clone just that tag rather than the full history:

```
git clone --branch v1.0 --depth 1 \
    https://github.com/GarethDaviesLondon/ValveInventory.git
cd ValveInventory
```

> **Note:** No release has been tagged yet at the time of writing - until v1.0 lands, use 2a (clone) or ask whoever gave you this manual for their own export (2c).

2c. You were given an exported .zip

Produced by File > Export archive and tools in the GUI. Just unzip it anywhere - it's not tied to git.

```
unzip valve_inventory_export.zip -d valve-inventory
cd valve-inventory
```

3. Building the database

The working database, `valves.db`, is never itself committed or exported - it's a binary SQLite file that would just bloat version control and can't be diffed. What travels with the project is a text snapshot in `data/`, which rebuilds it:

```
python3 snapshot.py --restore
```

This reads `data/valves.sql` and writes a fresh `valves.db` next to it. Safe to re-run any time - it will refuse to overwrite an existing database unless you pass `--force`.

4. Confirming it worked

```
python3 test_smoke.py
```

This runs a fast set of checks - the type-name classifier, a database round-trip, each CLI command against a scratch copy, and (if `data/` is present) the restore path itself. It should print `all checks passed` and exit cleanly. If it doesn't, see Troubleshooting below before going further.

5. Running it

Desktop window

```
python3 valves_gui.py
```

Command line

```
python3 valves.py stats
```

Both read and write the same `valves.db` - there is no separate setup for one versus the other.

6. The datasheet archive (optional)

PDF datasheets are not included in a clone or an export - hundreds of megabytes of third-party files that would swamp the repository. Build your own copy locally:

```
python3 fetch_datasheets.py --index      # maps the site - slow, run once, resumable
```

```
python3 fetch_datasheets.py --download   # pulls only the types this database holds
python3 valves.py scan                   # links the downloaded files in
```

The default 2-second delay between requests is deliberate - the source site runs on donations. Both stages are resumable, so Ctrl-C is always safe. If you'd rather have Claude do this work (including finding sources beyond that one site), the GUI's Tools > Generate datasheet download prompt... writes a ready-to-use prompt for an agent with file and web access, such as Claude Code.

7. Troubleshooting

Symptom	Likely cause / fix
“No module named tkinter”	Install the platform tkinter package (e.g. python3-tk on Debian/Ubuntu); it isn't installable via pip.
UnicodeDecodeError / UnicodeEncodeError during restore or CLI output	Windows consoles default to a non-UTF-8 codepage; some collection notes contain Cyrillic and other non-Latin text. This tool's own scripts already force UTF-8 - if you hit this in a modified copy, add encoding="utf-8" to the relevant open() call and sys.stdout.reconfigure(encoding="utf-8") near the top of main().
“valves.db already exists” on restore	Expected safety behaviour - pass --force if you intend to overwrite it, or delete/rename the existing file first if you're not sure what's in it.
GUI window closes immediately, no error visible	Run python3 valves_gui.py from a terminal instead of double-clicking the file, so any traceback stays visible after the window closes.
openpyxl / reportlab import errors	Both are optional - only needed for File > Export spreadsheet and for rebuilding these manuals respectively. pip install the missing one, or simply avoid that feature.

8. Starting your own collection

The database that ships in this repository is the author's own stock - real box locations in a real attic, not sample data. To make it yours instead:

Option A - start empty

Delete the working database and launch the app; a fresh, empty one is created automatically the moment anything tries to open it:

```
rm valves.db          # del valves.db on Windows
python3 valves_gui.py
```

Option B - keep the reference library, clear the stock

The researched valve types (function, base, heater, ratings) are useful on their own regardless of whose valves they are - keep that, wipe out the boxes and quantities that belong to the original owner's collection:

```
python3 -c "
import valvelib as V
con = V.init_db()
for t in ('stock', 'socket', 'sundry', 'box'):
    con.execute(f'DELETE FROM {t}')
con.commit()
n = con.execute('SELECT COUNT(*) FROM valve_type').fetchone()[0]
print('cleared - kept', n, 'reference types')
"
```

Either way, run `python3 snapshot.py` afterward if you want your own fork's `data/` to reflect the change before committing.

9. License and disclaimer

MIT-licensed - see the `LICENSE` file included in this repository. MIT is about as permissive as licenses get; its one real obligation, keeping the copyright notice attached, is also what gives it attribution. The data in `data/` carries its own, separate note in `LICENSE` - some of the descriptive text there is third-party material gathered from reference sites, not the author's to relicense.

> **Note:** This software is provided without warranty of any kind, express or implied, and you use it entirely at your own risk. It is hobbyist tooling built for one person's attic, not a certified reference - treat every “inferred” parameter as a lead to verify against a real datasheet, not a settled fact, especially before relying on it for anything involving the lethal voltages a valve amplifier runs at. By downloading, installing, or running this application, you are confirming that you have reviewed the source for yourself and that you accept these terms.